

Building a Convolutional Neural Network in PyTorch

Fashion-MNIST, Tensor Shapes, Training State, and Reliable Evaluation

Abstract

A convolutional neural network (CNN) preserves the two-dimensional structure of an image while learning local, reusable pattern detectors. These notes build a complete Fashion-MNIST classifier from a 60,000-row pixel table. The treatment begins with image tensors and a manual 3×3 cross-correlation, derives every convolution and pooling shape, explains channels, filters, Batch Normalization, ReLU, dropout, flattening, logits, and cross-entropy, and then maps the mathematics to runnable PyTorch code. The exact architecture transforms $[B, 1, 28, 28]$ image batches into $[B, 10]$ logits and contains 429,450 trainable parameters. A recorded 100-epoch configuration produced held-out accuracy 0.9261666666666667 and training accuracy 0.9999375; the resulting 7.37708-percentage-point gap is analyzed as overfitting rather than hidden by the high training score. The final sections establish training/evaluation state discipline, validation/test separation, executed shape and gradient checks, common failure modes, and review questions.

Contents

1	Purpose and complete mental map	2
1.1	The practical task	2
1.2	Project context and measurable goal	3
2	Image tensors, locality, and shared filters	4
2.1	Why an image is grid-structured data	4
2.2	Channels, filters, and feature maps	4
3	Convolution from expanded arithmetic to compact notation	5
3.1	A manual three-by-three calculation	5
3.2	The compact multi-channel operation	5
3.3	Output size and same padding	6
3.4	Parameter sharing in numbers	6
4	Pooling, activation, normalization, and dropout	7
4.1	Max pooling	7
4.2	ReLU	8
4.3	Batch Normalization	8
4.4	Dropout	8
5	Exact architecture and tensor-shape derivation	8
5.1	Shape flow	8
5.2	Parameter count	9
5.3	Object responsibilities before implementation	10

6	From a pixel table to image mini-batches	11
6.1	Imports, reproducibility, and device selection	11
6.2	Inspect the first 16 images	11
6.3	Split and scale	11
6.4	Dataset responsibility and reshape	12
6.5	DataLoader responsibility	12
7	Classifier logits and cross-entropy	13
8	Training algorithm and state transitions	14
8.1	Construct independent training state	14
8.2	One mini-batch update	15
8.3	Verified loss records	16
8.4	Evaluation without mutation	16
9	Measured results and overfitting interpretation	17
9.1	Accuracy as a count	17
9.2	Generalization gap	17
9.3	Training, validation, and test partitions	18
10	Executed reconstruction and reproducibility checks	18
11	Failure modes and debugging checklist	19
12	Review questions	20

1 Purpose and complete mental map

1.1 The practical task

Fashion-MNIST contains grayscale images of clothing and footwear. Each image has height 28 pixels, width 28 pixels, and one channel. A **pixel** is one numerical picture element. A **channel** is one aligned numerical grid; grayscale has one channel, whereas a conventional red–green–blue image has three.

A mini-batch of B images is stored as

$$X \in \mathbb{R}^{B \times 1 \times 28 \times 28}.$$

Here X is the input tensor, B is the number of examples processed together, 1 is the channel count, and the final two dimensions are height and width. The classifier must return

$$Z \in \mathbb{R}^{B \times 10},$$

where each row contains ten **logits**: unrestricted real-valued class scores. The class with the largest logit is the predicted category.

A flattened artificial neural network (ANN) can consume $[B, 784]$ because

$$28 \times 28 = 784.$$

This multiplication counts the pixels in one image. Flattening is valid, but it removes the explicit row–column organization before feature learning. A CNN keeps that organization and uses small shared filters to detect a pattern wherever it appears.

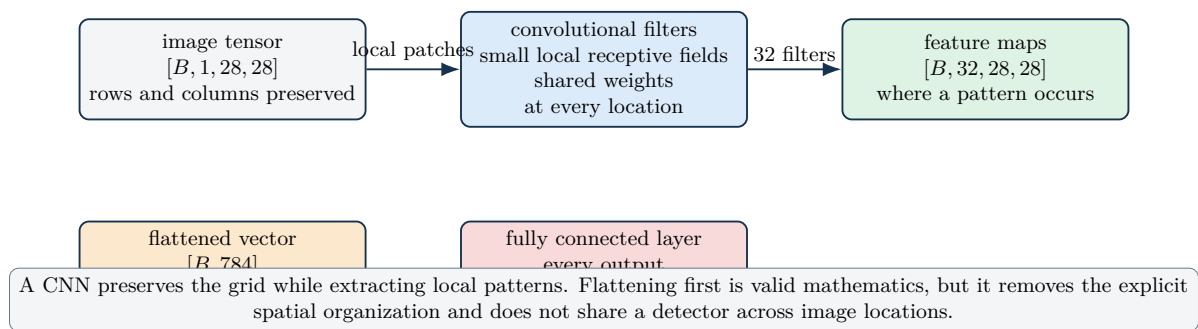
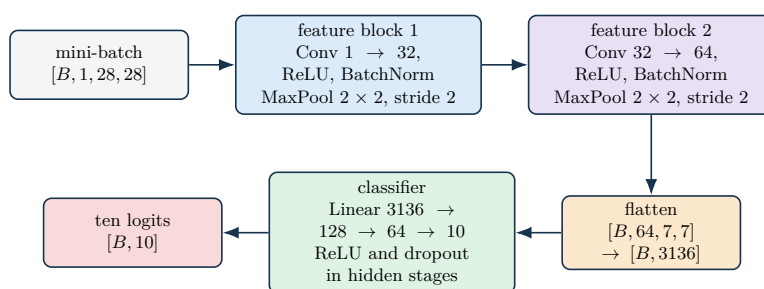


Figure 1: Convolution preserves the image grid and shares local detectors, whereas early flattening hides explicit spatial adjacency. [Technical reference](#).

Definition

A **convolutional neural network** is a neural-network architecture that learns local filters on grid-structured data. An image-classification CNN commonly contains a **feature extractor**, which produces spatial feature maps, and a **classifier**, which converts those maps into class logits.



Feature extraction converts pixels into spatial feature maps. The classifier then converts the final maps into one score per Fashion-MNIST class.

Figure 2: The complete model contains two convolution–pooling feature blocks followed by a fully connected classifier. [Technical reference](#).

1.2 Project context and measurable goal

Earlier flattened-ANN configurations provided the following comparison points.

Configuration	Accuracy	Interpretation
ANN on a CPU subset	83.2%	limited data and slow fitting
ANN on full data with a GPU	88.9%	higher score but strong overfitting
Regularized ANN	88.9%	similar held-out score with reduced overfitting
Hyperparameter-tuned ANN	90.2%	strongest reported flattened-ANN configuration

The immediate goal is not merely to exceed 90.2%. A useful model should also maintain a small difference between training and unseen-data performance. CNN optimization and transfer learning with pretrained architectures such as ResNet or VGG are later extensions; the present model establishes the basic PyTorch CNN pipeline first.

Takeaway

The architecture must preserve image shape through feature extraction, maintain a valid tensor contract at every layer, and evaluate generalization separately from memorization.

2 Image tensors, locality, and shared filters

2.1 Why an image is grid-structured data

Two pixels can be numerically similar yet play different roles because of their locations. Nearby pixels often form edges, corners, textures, and parts. A **spatial pattern** is a relationship among values at nearby row-column positions. Image classification, recognition, and object detection all benefit from preserving such relationships.

A CNN encodes two useful assumptions:

Locality. A small neighborhood is sufficient to detect many low-level patterns such as an edge.

Translation reuse. A useful detector should be reusable at the left, center, or right of an image rather than owning separate weights at every position.

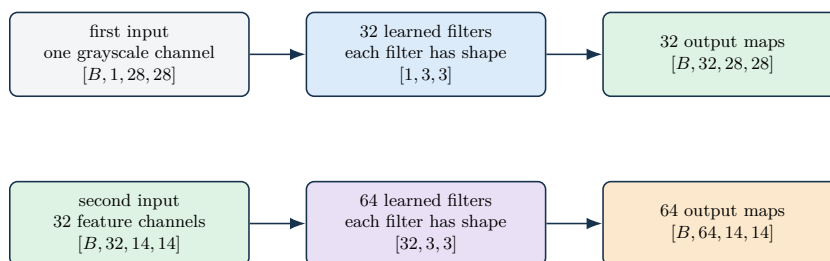
These assumptions form an **inductive bias**: a structural preference built into the model before fitting. They do not guarantee that a CNN always wins, but they match many image tasks better than an immediate fully connected map.

2.2 Channels, filters, and feature maps

For a PyTorch image tensor $[B, C, H, W]$:

- B is batch size;
- C is channel count;
- H is height;
- W is width.

A convolutional **filter** or **kernel** is a small learned tensor. One filter spans every input channel and produces one output channel, also called one **feature map**. Therefore, the first layer uses 32 filters of shape $[1, 3, 3]$, while each of the 64 second-layer filters has shape $[32, 3, 3]$.



`in_channels` must equal the preceding tensor's channel count. `out_channels` equals the number of filters and therefore the number of output maps.

Figure 3: Input channels determine each filter's depth; the number of filters determines the output-channel count. [Technical reference](#).

Shape and state check

For every `Conv2d`, `in_channels` must equal the preceding tensor's channel count. A second convolution receiving 32 feature maps must use `in_channels=32`; using 1 or 3 violates the contract.

3 Convolution from expanded arithmetic to compact notation

3.1 A manual three-by-three calculation

Consider the image patch

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

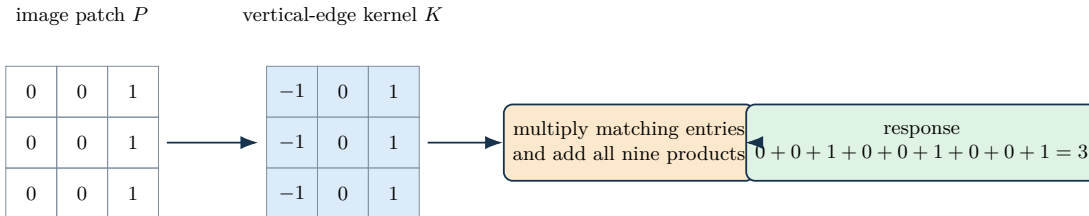
and a vertical-edge kernel

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}.$$

The operation multiplies matching entries and adds all nine products:

$$\begin{aligned} r &= (0)(-1) + (0)(0) + (1)(1) \\ &\quad + (0)(-1) + (0)(0) + (1)(1) \\ &\quad + (0)(-1) + (0)(0) + (1)(1) \\ &= 3. \end{aligned}$$

The scalar r is the filter response at this location. A positive value indicates agreement with this filter's orientation. During neural-network training, the entries of K are learned rather than fixed to these hand-selected values.



PyTorch `Conv2d` computes cross-correlation: the stored kernel is not flipped. Sliding the same K across all valid locations shares one detector and produces one feature map. A learned filter replaces these hand-selected values during training.

Figure 4: One cross-correlation response is the sum of nine elementwise products; the executed example equals 3. [Technical reference](#).

PyTorch's operation is technically **cross-correlation**: it applies the stored kernel without flipping it. Deep-learning practice commonly calls the layer convolution because the learned-kernel behavior is functionally the same for model construction.

3.2 The compact multi-channel operation

For a complete batch, one output value can be written as

$$Y_{b,o,i,j} = \beta_o + \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} W_{o,c,u,v} X_{b,c,iS_h+u-P_h,jS_w+v-P_w}.$$

Every symbol has a defined role:

- X is the input tensor and Y is the output tensor;
- b selects one batch example;
- o selects one output filter;
- (i, j) selects one output row and column;
- c selects one input channel;
- (u, v) selects one kernel row and column;
- $W_{o,c,u,v}$ is one learned filter weight;
- β_o is the learned bias for output channel o ;
- C_{in} is the number of input channels;
- K_h, K_w are kernel height and width;
- S_h, S_w are strides;
- P_h, P_w are padding amounts.

The equation computes one weighted local sum. Repeating it across i, j, o , and b creates all feature maps for the batch. It matters because the same weights $W_{o,c,u,v}$ are reused at every spatial location.

3.3 Output size and same padding

For one spatial dimension, convolution output size is

$$L_{\text{out}} = \left\lfloor \frac{L_{\text{in}} + 2P - D(K - 1) - 1}{S} + 1 \right\rfloor.$$

Here L_{in} is input size, P is padding, D is dilation, K is kernel size, and S is stride. The floor symbol discards any fractional remainder. For $L_{\text{in}} = 28$, $P = 1$, $D = 1$, $K = 3$, and $S = 1$,

$$L_{\text{out}} = \left\lfloor \frac{28 + 2 - 2 - 1}{1} + 1 \right\rfloor = 28.$$

Thus kernel 3 with one padded cell on each side preserves height and width. In code, `padding="same"` requests this preservation for the stride-1 convolutions.

3.4 Parameter sharing in numbers

The first convolution has 32 filters. Each filter owns $1 \times 3 \times 3 = 9$ weights and one bias:

$$P_{\text{conv1}} = 32(1 \cdot 3 \cdot 3 + 1) = 320.$$

The symbol P_{conv1} counts trainable scalars in the first convolution. By comparison, a dense $784 \rightarrow 128$ layer owns

$$P_{784 \rightarrow 128} = 784 \cdot 128 + 128 = 100,480$$

trainable scalars. These counts do not compare whole models; they isolate why one reusable local detector can be far more parameter-efficient than a separate weight for every input–output pair.

```
import torch
import torch.nn.functional as F

patch = torch.tensor([[[[
    [0.0, 0.0, 1.0],
    [0.0, 0.0, 1.0],
    [0.0, 0.0, 1.0],
]]]])
kernel = torch.tensor([[[[
    [-1.0, 0.0, 1.0],
    [-1.0, 0.0, 1.0],
    [-1.0, 0.0, 1.0],
]]]])

response = F.conv2d(patch, kernel)
assert response.item() == 3.0
assert 32 * (1 * 3 * 3 + 1) == 320
assert 784 * 128 + 128 == 100_480
```

Listing 1: Verify the manual cross-correlation and parameter-sharing example

4 Pooling, activation, normalization, and dropout

4.1 Max pooling

Max pooling keeps the largest activation in a local window. For the window

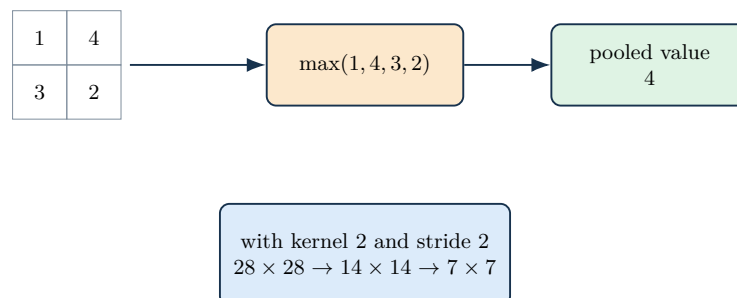
$$M = \begin{bmatrix} 1 & 4 \\ 3 & 2 \end{bmatrix}, \quad \text{MaxPool}(M) = \max(1, 4, 3, 2) = 4.$$

The matrix M contains four neighboring activations. The result 4 preserves the strongest response and discards the other three values. With kernel size 2 and stride 2, non-overlapping windows halve both height and width:

$$28 \rightarrow 14 \rightarrow 7.$$

Pooling changes spatial resolution but not channel count.

2×2 pooling window



Max pooling keeps the largest activation in each non-overlapping window. It reduces spatial resolution and computation, but it does not change the number of channels.

Figure 5: A 2×2 max-pool operation returns the largest value and halves each spatial dimension when stride is 2. [Technical reference](#).

```

window = torch.tensor([[[[1.0, 4.0], [3.0, 2.0]]]])
pooled = F.max_pool2d(window, kernel_size=2, stride=2)
assert pooled.shape == (1, 1, 1, 1)
assert pooled.item() == 4.0

```

Listing 2: Verify one max-pooling window

4.2 ReLU

The rectified linear unit (ReLU) is

$$\text{ReLU}(a) = \max(0, a).$$

The symbol a is one pre-activation scalar. Negative values become zero and positive values remain unchanged. ReLU introduces nonlinearity; without nonlinear activation functions, stacked affine layers collapse into one affine transformation.

4.3 Batch Normalization

For one channel, a simplified training-time Batch Normalization operation is

$$\hat{a} = \frac{a - \mu_{\mathcal{B}}}{(\sigma_{\mathcal{B}}^2 + \varepsilon)^{1/2}}, \quad q = \gamma \hat{a} + \beta.$$

Here a is an activation, $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the mini-batch mean and variance for that channel, ε prevents division by zero, and γ and β are learned scale and shift parameters. The normalized output q can improve optimization stability. `BatchNorm2d(32)` owns one γ and one β for each of 32 channels; `BatchNorm2d(64)` needs 64 of each.

The implemented order is `Conv` $\rightarrow$ `ReLU` $\rightarrow$ `BatchNorm` $\rightarrow$ `Pool`. `Conv` $\rightarrow$ `BatchNorm` $\rightarrow$ `ReLU` $\rightarrow$ `Pool` is also common. The order is a design choice; it should be recorded and compared rather than silently changed.

4.4 Dropout

For a hidden activation h_i , inverted dropout can be written as

$$\tilde{h}_i = \frac{m_i}{1-p} h_i, \quad m_i \sim \text{Bernoulli}(1-p).$$

The symbol p is the probability of dropping a unit, and m_i equals 0 with probability p or 1 with probability $1-p$. Division by $1-p$ preserves the expected activation scale during training. The model uses $p = 0.4$ after both hidden ReLU operations. During evaluation, dropout is disabled.

Pitfall

Dropout and Batch Normalization depend on model state. Calling only a no-gradient context does not disable dropout or freeze Batch Normalization statistics; `model.eval()` is also required.

5 Exact architecture and tensor-shape derivation

5.1 Shape flow

The complete shape path is

$$\begin{aligned}
[B, 1, 28, 28] &\rightarrow [B, 32, 28, 28] \rightarrow [B, 32, 14, 14] \\
&\rightarrow [B, 64, 14, 14] \rightarrow [B, 64, 7, 7] \\
&\rightarrow [B, 3136] \rightarrow [B, 128] \rightarrow [B, 64] \rightarrow [B, 10].
\end{aligned}$$

Convolutions change channel count while same padding preserves spatial size. Pools preserve channel count while halving spatial size. Flattening preserves the batch dimension and multiplies the remaining dimensions:

$$64 \cdot 7 \cdot 7 = 3136.$$

The linear head then changes feature width without changing B .

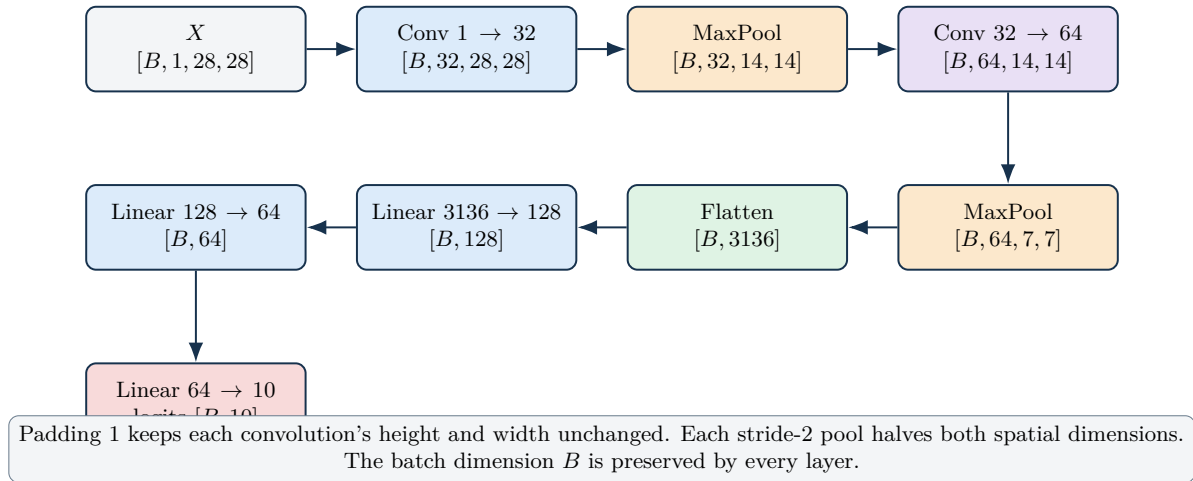


Figure 6: Every feature and classifier stage preserves batch size while changing channels, spatial size, or feature width. [Technical reference](#).

Operation	Configuration	Output shape	Responsibility
Input	grayscale images	$[B, 1, 28, 28]$	preserve batch, channel, height, width
Conv2d	$1 \rightarrow 32$, 3×3 , same	$[B, 32, 28, 28]$	learn 32 local maps
ReLU + BatchNorm2d	32 channels	$[B, 32, 28, 28]$	nonlinear activation and channel normalization
MaxPool2d	2×2 , stride 2	$[B, 32, 14, 14]$	halve spatial resolution
Conv2d	$32 \rightarrow 64$, 3×3 , same	$[B, 64, 14, 14]$	learn 64 deeper maps
ReLU + BatchNorm2d	64 channels	$[B, 64, 14, 14]$	nonlinear activation and channel normalization
MaxPool2d	2×2 , stride 2	$[B, 64, 7, 7]$	halve spatial resolution
Flatten	$64 \cdot 7 \cdot 7$	$[B, 3136]$	convert maps to one vector per image
Linear + ReLU + Dropout	$3136 \rightarrow 128$	$[B, 128]$	first hidden classifier stage
Linear + ReLU + Dropout	$128 \rightarrow 64$	$[B, 64]$	second hidden classifier stage
Linear	$64 \rightarrow 10$	$[B, 10]$	produce class logits

5.2 Parameter count

The trainable parameter count is

$$\begin{aligned}
 P &= 32(1 \cdot 3 \cdot 3 + 1) && \text{first convolution} \\
 &+ 2(32) && \text{first BatchNorm scale and shift} \\
 &+ 64(32 \cdot 3 \cdot 3 + 1) && \text{second convolution} \\
 &+ 2(64) && \text{second BatchNorm scale and shift} \\
 &+ (3136 \cdot 128 + 128) && \text{first linear layer} \\
 &+ (128 \cdot 64 + 64) && \text{second linear layer} \\
 &+ (64 \cdot 10 + 10) && \text{output layer} \\
 &= 320 + 64 + 18,496 + 128 \\
 &\quad + 401,536 + 8,256 + 650 \\
 &= 429,450.
 \end{aligned}$$

The symbol P counts trainable scalar weights and biases. ReLU, MaxPool, Flatten, and Dropout have no trainable parameters. Batch Normalization running means and variances are buffers rather than trainable parameters, so they are not added to P .

5.3 Object responsibilities before implementation

The class inherits `nn.Module`, which manages child-module registration, parameter traversal, device movement, and training/evaluation state. The constructor creates two containers:

features preserves spatial structure and returns $[B, 64, 7, 7]$.

classifier flattens each example and returns $[B, 10]$ logits.

Calling `super().__init__()` before assigning child modules is an invariant; otherwise PyTorch cannot register those modules correctly.

```
import torch.nn as nn

class BasicCNN(nn.Module):
    def __init__(self, input_channels=1):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(
                input_channels, 32,
                kernel_size=3, padding="same"
            ),
            nn.ReLU(),
            nn.BatchNorm2d(32),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(
                32, 64, kernel_size=3, padding="same"
            ),
            nn.ReLU(),
            nn.BatchNorm2d(64),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, 128),
            nn.ReLU(),
            nn.Dropout(p=0.4),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Dropout(p=0.4),
            nn.Linear(64, 10),
        )

    def forward(self, features):
        features = self.features(features)
        return self.classifier(features)
```

Listing 3: Define the complete convolutional classifier

Shape and state check

The second normalization layer must be `BatchNorm2d(64)` because the preceding convolution emits 64 channels. Copying `BatchNorm2d(32)` into that position causes a channel-state mismatch.

6 From a pixel table to image mini-batches

6.1 Imports, reproducibility, and device selection

The table contains 60,000 examples, one integer label column, and 784 pixel columns. The setup uses seed 42 and selects a CUDA device when available.

```
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from torch.utils.data import Dataset, DataLoader

torch.manual_seed(42)
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
print(f"Using device: {device}")

frame = pd.read_csv("fashion-mnist_train.csv")
assert frame.shape == (60_000, 785)
```

Listing 4: Import dependencies, choose a device, and load the pixel table

A seed constrains some random operations but does not by itself guarantee identical metrics across hardware, library versions, loader worker schedules, and nondeterministic kernels.

6.2 Inspect the first 16 images

A tabular row must be reshaped before plotting. The label is in the first column and the remaining 784 values form the image.

```
fig, axes = plt.subplots(4, 4, figsize=(10, 10))
fig.suptitle("First 16 Images", fontsize=16)

for row_index, axis in enumerate(axes.flat):
    image = frame.iloc[row_index, 1:].to_numpy()
    image = image.reshape(28, 28)
    label = int(frame.iloc[row_index, 0])
    axis.imshow(image, cmap="gray")
    axis.set_title(f"Label: {label}")
    axis.axis("off")

plt.tight_layout()
plt.show()
```

Listing 5: Render a four-by-four grid of table rows as grayscale images

The grid is a data-integrity check: images should have plausible clothing shapes and labels should be integer category identifiers. Plotting is not part of model training.

6.3 Split and scale

The 20% held-out split contains

$$0.2 \cdot 60,000 = 12,000$$

examples, leaving 48,000 for weight updates. Dividing byte-valued pixels by 255 maps the range $[0, 255]$ to $[0, 1]$ without changing shape.

```

features = frame.iloc[:, 1:].to_numpy()
labels = frame.iloc[:, 0].to_numpy()

X_train, X_held, y_train, y_held = train_test_split(
    features,
    labels,
    test_size=0.2,
    random_state=42,
)
X_train = X_train.astype("float32") / 255.0
X_held = X_held.astype("float32") / 255.0

assert X_train.shape == (48_000, 784)
assert X_held.shape == (12_000, 784)
assert y_train.shape == (48_000,)
assert y_held.shape == (12_000,)

```

Listing 6: Separate columns, split rows, and scale pixel values

6.4 Dataset responsibility and reshape

A `Dataset` maps one integer index to one aligned image–label pair. Its central invariant is equal feature and label length. The constructor reshapes all rows at once:

$$[N, 784] \rightarrow [N, 1, 28, 28],$$

where N is the number of examples in that partition. The leading -1 in code asks PyTorch to infer N from the total element count; it does not mean batch size 32.

```

class CustomDataset(Dataset):
    def __init__(self, features, labels):
        self.features = torch.tensor(
            features, dtype=torch.float32
        ).reshape(-1, 1, 28, 28)
        self.labels = torch.tensor(
            labels, dtype=torch.long
        )
        if len(self.features) != len(self.labels):
            raise ValueError("feature/label length mismatch")

    def __len__(self):
        return len(self.features)

    def __getitem__(self, index):
        return self.features[index], self.labels[index]

```

Listing 7: Store aligned image tensors and integer class labels

One returned image has shape $[1, 28, 28]$ and one returned label is a scalar `torch.int64` value. Cross-entropy requires integer class indices rather than floating one-hot vectors for this usage.

6.5 DataLoader responsibility

A `DataLoader` chooses indices, retrieves individual examples, and stacks them into mini-batches. For batch size 32 it returns images $[32, 1, 28, 28]$ and labels $[32]$.

```

train_dataset = CustomDataset(X_train, y_train)
held_dataset = CustomDataset(X_held, y_held)

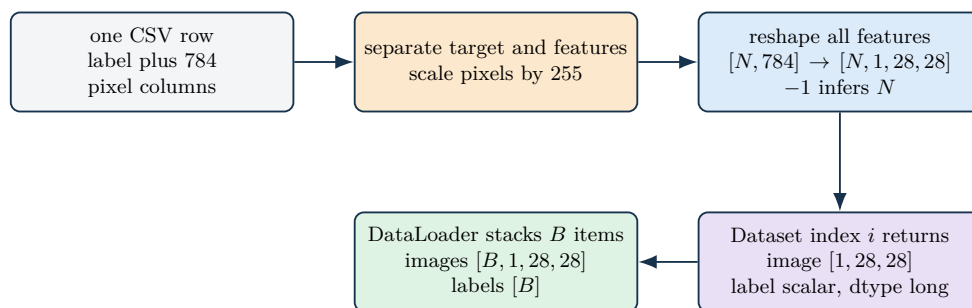
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,
    pin_memory=(device.type == "cuda"),
)
held_loader = DataLoader(
    held_dataset,
    batch_size=32,
    shuffle=False,
    pin_memory=(device.type == "cuda"),
)

images, targets = next(iter(train_loader))
assert images.shape == (32, 1, 28, 28)
assert targets.shape == (32,)

```

Listing 8: Create training and held-out mini-batch iterators

Pinned host memory can improve compatible CPU-to-CUDA transfer. It does not move a tensor to a GPU by itself. Explicit `to(device)` calls remain necessary.



The leading batch dimension belongs to the loader. A single stored example has no batch axis; stacking examples creates B .

Figure 7: A flat table row becomes an image-shaped Dataset item, and DataLoader stacking creates the batch dimension. [Technical reference](#).

Pitfall

Calling `reshape(1, 28, 28)` on the entire $[N, 784]$ table is impossible because it omits N . Calling the CNN directly on $[B, 784]$ also fails because `Conv2d` expects four dimensions $[B, C, H, W]$.

7 Classifier logits and cross-entropy

For logits $z_{i,c}$, the softmax probability for example i and class c is

$$p_{i,c} = \frac{\exp(z_{i,c})}{\sum_{d=0}^9 \exp(z_{i,d})}.$$

Here $z_{i,c}$ is one unrestricted score, $p_{i,c}$ is a normalized probability, and d indexes all ten classes. The denominator makes the ten probabilities sum to 1.

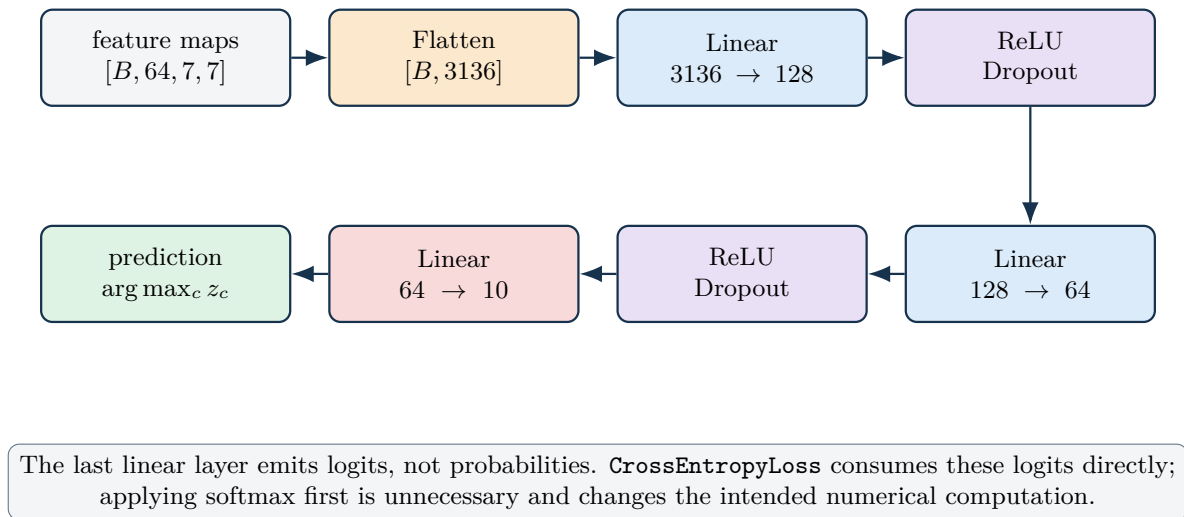


Figure 8: The classifier flattens final feature maps, applies two regularized hidden stages, and emits ten logits. [Technical reference](#).

For a batch of B integer labels y_i , mean cross-entropy is

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log p_{i,y_i}.$$

The symbol $\mathcal{L}$ is the scalar loss, and p_{i,y_i} is the probability assigned to the correct class. A confident correct prediction makes this probability near 1 and contributes a value near 0. A confident wrong prediction contributes a large positive penalty.

For one example with logits $[2, 1, 0]$ and correct class 0,

$$\exp(2) = 7.3891, \quad \exp(1) = 2.7183, \quad \exp(0) = 1,$$

so the sum is 11.1073. The correct-class probability and loss are

$$p_0 = \frac{7.3891}{11.1073} = 0.66524, \quad -\log(p_0) = 0.40761.$$

This expanded calculation is what the compact equations perform. PyTorch’s `CrossEntropyLoss` combines a numerically stable log-softmax with negative log-likelihood. The model should return logits directly; adding softmax before the criterion is unnecessary and alters the intended computation.

Shape and state check

For B examples and ten classes, logits must have shape $[B, 10]$. Targets must have shape $[B]$ and dtype `torch.long`. A target tensor shaped $[B, 1]$ or stored as floating-point class indices violates this contract.

8 Training algorithm and state transitions

8.1 Construct independent training state

The model, criterion, and optimizer have distinct responsibilities:

Model owns trainable parameters and computes logits.

Criterion compares logits with labels and returns scalar loss.

Optimizer reads gradients and mutates parameters.

The reported configuration uses 100 epochs, learning rate $\eta = 0.01$, stochastic gradient descent (SGD), and weight decay 10^{-4} .

```
learning_rate = 0.01
epochs = 100

model = BasicCNN(input_channels=1).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(
    model.parameters(),
    lr=learning_rate,
    weight_decay=1e-4,
)
```

Listing 9: Create model, criterion, and optimizer state

The learning-rate variable must reach the optimizer. Assigning `learning_rate=0.01` while hard-coding `lr=0.1` creates an inert variable and a misleading experiment. Passing `lr=learning_rate` keeps configuration and effective behavior aligned.

8.2 One mini-batch update

For parameter vector θ and learning rate η , a simplified SGD update is

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_t.$$

The symbol t indexes optimizer steps, $\nabla_{\theta} \mathcal{L}_t$ is the loss gradient with respect to every trainable parameter, and θ_{t+1} is the updated parameter state. Weight decay adds a shrinkage-related term according to the optimizer's implementation.

The code order mirrors the equation:

1. clear gradients from the preceding step;
2. compute logits and loss;
3. call backward to compute $\nabla_{\theta} \mathcal{L}_t$;
4. call the optimizer to mutate θ_t .

```
for epoch in range(epochs):
    model.train()
    total_epoch_loss = 0.0

    for images, targets in train_loader:
        images = images.to(device)
        targets = targets.to(device)

        optimizer.zero_grad(set_to_none=True)
        logits = model(images)
        loss = criterion(logits, targets)
        loss.backward()
        optimizer.step()

        total_epoch_loss += loss.item()

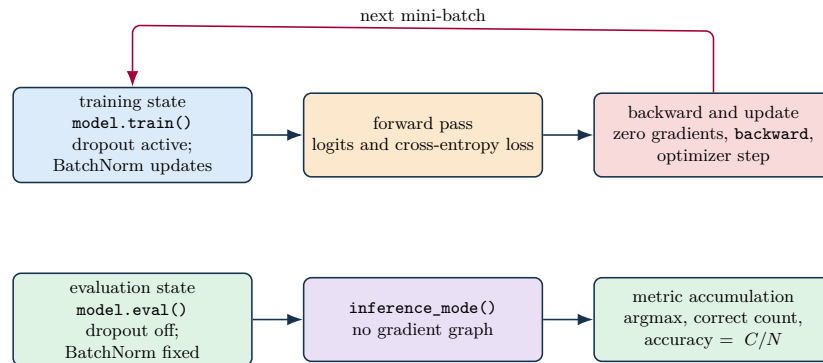
    average_loss = total_epoch_loss / len(train_loader)
    print(f"Epoch: {epoch + 1}, Loss: {average_loss}")
```

Listing 10: Train for 100 epochs and report mean mini-batch loss

If M is the number of mini-batches and ℓ_m is the scalar loss from batch m , the printed mean is

$$\bar{\ell} = \frac{1}{M} \sum_{m=1}^M \ell_m.$$

The symbol $\bar{\ell}$ is the unweighted mean of batch means. Because every full batch has equal size here, it closely represents mean example loss; a smaller final batch would require example-weighted aggregation for exactness.



Mode switching and gradient disabling solve different problems. Correct inference ordinarily uses both `eval()` and an inference/no-gradient context.

Figure 9: Training activates dropout, Batch Normalization updates, gradients, and optimizer mutation; evaluation freezes these behaviors. [Technical reference](#).

8.3 Verified loss records

One run printed first-epoch mean loss 0.6453862568239371. The final ten values were:

Epoch	Mean loss	Epoch	Mean loss
91	0.015572583074495924	96	0.016346150235516537
92	0.017386720219790125	97	0.013636079695961294
93	0.018627057177018288	98	0.013638910541678645
94	0.0161921720666669	99	0.017204439169262665
95	0.015050439908358991	100	0.013950658754194288

The late values fluctuate because SGD sees shuffled mini-batches and dropout samples random masks. A noisy increase from one epoch to the next does not by itself imply failure; trends in training and validation metrics are more informative than monotonicity of a single stochastic trace.

8.4 Evaluation without mutation

Evaluation switches layer behavior and disables gradient bookkeeping before counting predictions.


```
def accuracy(model, loader, device):
    model.eval()
    correct = 0
    total = 0

    with torch.inference_mode():
        for images, targets in loader:
            images = images.to(device)
            targets = targets.to(device)
            logits = model(images)
            predictions = logits.argmax(dim=1)
            correct += (predictions == targets).sum().item()
            total += targets.numel()

    return correct / total

held_accuracy = accuracy(model, held_loader, device)
train_accuracy = accuracy(model, train_loader, device)
```

Listing 11: Compute accuracy without changing model state

`model.eval()` changes dropout and Batch Normalization. `torch.inference_mode()` disables autograd bookkeeping. These operations solve different problems, so correct inference ordinarily uses both.

9 Measured results and overfitting interpretation

9.1 Accuracy as a count

For N examples and C correct predictions,

$$A = \frac{C}{N}.$$

The symbol A is accuracy. The held-out print was

$$A_{\text{held}} = 0.9261666666666667 \approx 92.6167\%.$$

With $N_{\text{held}} = 12,000$, this value corresponds to

$$C_{\text{held}} = 11,114, \quad 12,000 - 11,114 = 886$$

correct and incorrect predictions, respectively.

Training accuracy was

$$A_{\text{train}} = 0.9999375 = 99.99375\%,$$

which corresponds to 47,997 correct predictions out of 48,000 and only 3 errors on the fitted partition.

9.2 Generalization gap

The absolute training–held-out difference is

$$G = A_{\text{train}} - A_{\text{held}} = 0.07377083333333333,$$

or approximately 7.37708 percentage points. The symbol G is the **generalization gap**. A large positive gap indicates that training performance is much stronger than unseen-data performance. It is evidence of overfitting even though the held-out score is useful.

Compared with the tuned-ANN value 90.2%, the basic CNN held-out score improved by

$$92.6167 - 90.2 \approx 2.4167$$

percentage points. This comparison supports the value of preserving image structure for this experiment, but it does not isolate architecture as the only cause unless data split, seed, preprocessing, optimizer budget, and evaluation protocol are controlled.

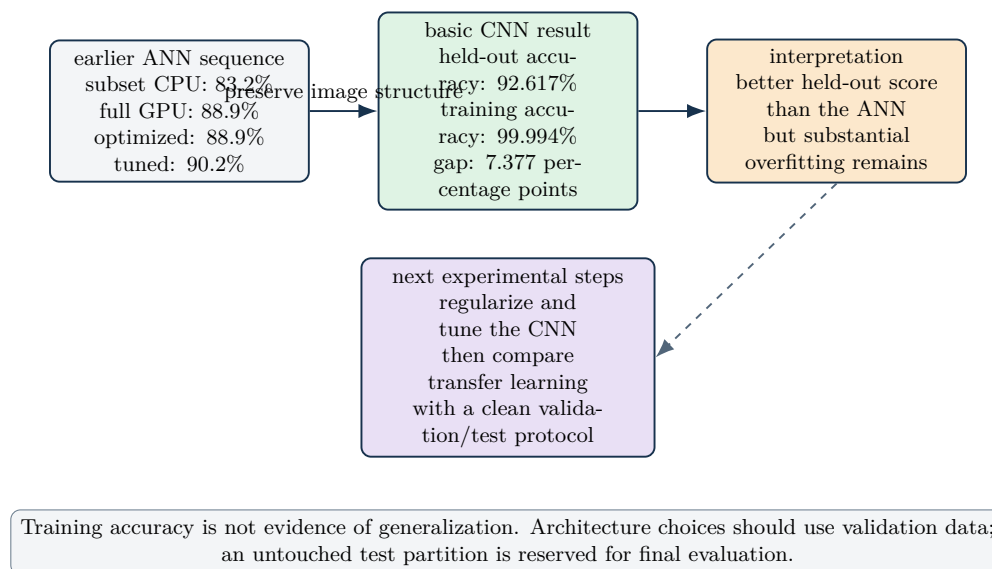


Figure 10: The basic CNN improves the held-out score while its near-perfect training score reveals substantial overfitting. [Technical reference](#).

9.3 Training, validation, and test partitions

Partition names do not determine their statistical role. If a held-out split influences architecture, dropout, epoch count, or any other development choice, it functions as **validation data**. A defensible workflow is:

Training partition update weights, biases, optimizer state, and Batch Normalization statistics.

Validation partition compare architectures and choose development settings.

Test partition remain untouched until all decisions are frozen, then estimate final generalization once.

Repeatedly choosing a model because it scores best on a variable named `test_loader` leaks information from that partition. Renaming the variable does not repair the protocol; information flow determines the role.

Takeaway

The CNN's 92.6167% held-out accuracy is stronger than the flattened-ANN comparison, but 99.99375% training accuracy exposes a large gap. The next experiments should reduce that gap using a true validation partition, regularization, tuning, data augmentation, and eventually transfer learning.

10 Executed reconstruction and reproducibility checks

An isolated CPU execution verified the complete architecture with PyTorch 2.12.0. It downloaded the canonical 60,000-image Fashion-MNIST training collection, reproduced a 48,000/12,000 split, and ran a bounded one-epoch smoke experiment on 512 training and 128 held-out examples.

The execution established:

- the exact output shape `[32, 10]` for input `[32, 1, 28, 28]`;
- the full intermediate shape trace through all eight measured stages;
- exactly 429,450 trainable parameters;
- manual cross-correlation response 3 and max-pool response 4;

- finite cross-entropy, backward propagation, and a nonzero SGD parameter update;
- deterministic repeated outputs in evaluation state;
- different repeated outputs in training state because dropout is active;
- six expected shape, dtype, and state failures.

The bounded Fashion-MNIST smoke run achieved 20 correct predictions out of 128, or 0.15625, after one CPU epoch. That value verifies end-to-end execution only; it is not a replacement for the 100-epoch result. CUDA was unavailable locally, so no GPU timing or full 100-epoch reproduction is claimed.

```
probe = BasicCNN(input_channels=1)
probe.eval()

with torch.inference_mode():
    logits = probe(torch.zeros(32, 1, 28, 28))

parameter_count = sum(
    parameter.numel() for parameter in probe.parameters()
)
assert logits.shape == (32, 10)
assert parameter_count == 429_450
assert probe.features(torch.zeros(32, 1, 28, 28)).shape == (
    32, 64, 7, 7
)
```

Listing 12: Make central shape and parameter invariants executable

Reproducing exact metrics additionally requires the same 60,000 table rows, split, loader order, random states, software stack, device kernels, optimizer state, and all 100 epochs. A fixed seed is necessary context but not a universal guarantee of bitwise equality.

11 Failure modes and debugging checklist

Flattened input reaches Conv2d. Supply $[B, C, H, W]$, not $[B, 784]$; reshape the Dataset features before batching.

Wrong channel count. A model constructed with `input_channels=1` rejects $[B, 3, 28, 28]$ RGB tensors.

Wrong reshape. Use `reshape(-1, 1, 28, 28)` for the whole table or `reshape(1, 28, 28)` for one row; do not mix these contexts.

Wrong BatchNorm width. After the $32 \rightarrow 64$ convolution, use `BatchNorm2d(64)`. A copied value 32 mismatches the tensor.

Wrong flatten width. Two stride-2 pools produce 7×7 , so the first linear layer requires $64 \cdot 7 \cdot 7 = 3136$ inputs.

Changed image size. Hard-coded 3136 is valid only for the derived 28×28 pipeline. Recompute it or use a carefully initialized lazy/adaptive design when dimensions change.

Float class indices. Cross-entropy class targets must be `torch.long`, not floating point.

Column-vector labels. Use labels shaped $[B]$, not $[B, 1]$.

Softmax before cross-entropy. Return logits directly. The criterion already performs the stable log-softmax calculation.

Inert learning-rate variable. Pass `lr=learning_rate`; do not assign 0.01 and silently hard-code 0.1 elsewhere.

Gradients accumulate. Clear gradients before every backward pass with `zero_grad`.

Evaluation remains in training state. Call `model.eval()` so dropout is disabled and Batch Normalization uses stored statistics.

- Autograd remains enabled.** Use `inference_mode()` or `no_grad()` during metrics to avoid unnecessary graph construction.
- Device mismatch.** Model, images, and targets must be on compatible devices before the forward pass and loss calculation.
- Assuming pinned memory moves data.** Pinned memory can assist transfer; explicit `to(device)` still performs the move.
- Shuffled held-out loader.** Shuffling does not change aggregate accuracy, but disabling it gives stable ordering for diagnostics and per-example analysis.
- Training accuracy used as success evidence.** Near-perfect fitting can coexist with weaker generalization; inspect validation curves and the gap.
- Development data called test data.** If it guides a decision, it is validation data. Reserve an independent test partition.
- Loss expected to be monotone.** Mini-batch order and dropout make epoch loss noisy. Judge trends and validation behavior, not every local increase.
- Architecture comparison is uncontrolled.** Keep split, preprocessing, seed policy, optimizer budget, and evaluation procedure comparable before attributing improvement to CNN structure alone.

12 Review questions

1. Define pixel, channel, filter, feature map, logit, mini-batch, and spatial pattern.
2. Why does flattening 28×28 to 784 hide explicit adjacency even though no values are deleted?
3. Reproduce the nine products that make the manual edge response equal 3.
4. Define every index and dimension in the multi-channel cross-correlation equation.
5. Use the convolution output-size formula to show why kernel 3, stride 1, and padding 1 preserve size 28.
6. Why does a $1 \rightarrow 32$ convolution use filters shaped $[1, 3, 3]$, while a $32 \rightarrow 64$ convolution uses filters shaped $[32, 3, 3]$?
7. Explain why 64 second-layer filters produce 64 output feature maps.
8. Compute the max-pool result for $\begin{bmatrix} 1 & 4 \\ 3 & 2 \end{bmatrix}$ and explain the $28 \rightarrow 14 \rightarrow 7$ size path.
9. Derive every tensor shape from $[B, 1, 28, 28]$ to $[B, 10]$.
10. Why is the flatten width 3136 rather than 784, 64, or 4096?
11. Reproduce all seven terms in the 429,450 parameter count.
12. What responsibilities belong to `Dataset`, `DataLoader`, `nn.Module`, the criterion, and the optimizer?
13. Why does `reshape(-1, 1, 28, 28)` infer the number of examples rather than the mini-batch size?
14. Explain why labels must be shape $[B]$ with dtype `torch.long`.
15. Convert logits $[2, 1, 0]$ into probabilities and reproduce the correct-class loss 0.40761.
16. Why should `CrossEntropyLoss` receive logits rather than softmax probabilities?
17. Distinguish `model.eval()` from `torch.inference_mode()`.
18. Map `zero_grad`, forward, loss, backward, and `step` to the SGD update equation.
19. Use the exact training and held-out accuracies to reproduce the 7.37708-percentage-point gap.

20. Why does repeated architecture selection make a held-out partition validation data, regardless of its variable name?
21. Design a three-way train–validation–test experiment for comparing this CNN with a tuned ANN.
22. Which changes would be necessary for RGB 32×32 images?
23. Propose two controlled experiments to reduce overfitting without hiding the held-out result.
24. Explain what the one-epoch CPU smoke run verifies and what it does not establish.